

Lesson 06: Terraform Variables

Learning Objectives

- Understand input variables and why they are useful
 - Define variables with type constraints and default values
 - Use variables in resource configurations
 - Pass variable values via `terraform.tfvars`, environment variables, and CLI flags
 - Understand variable validation and sensitivity
-

1. What are Input Variables?

Input variables make your Terraform configurations **parameterized** and **reusable**. Instead of hardcoding values like bucket names and regions, you define variables and pass different values for different environments.

```
# Without variables (hardcoded)
resource "aws_s3_bucket" "main" {
  bucket = "my-prod-bucket" # <-- hardcoded
}

# With variables (parameterized)
variable "bucket_name" {
  description = "Name of the S3 bucket"
  type        = string
}

resource "aws_s3_bucket" "main" {
  bucket = var.bucket_name # <-- referenced
}
```

2. Defining Variables

Variables are defined in `.tf` files using the `variable` block:

```
variable "environment" {
  description = "Deployment environment"
  type        = string
  default     = "dev"
}
```

Variable Block Arguments

Argument	Description	Required
<code>description</code>	Human-readable explanation of the variable	No
<code>type</code>	Type constraint (<code>string</code> , <code>number</code> , <code>bool</code> , <code>list</code> , <code>map</code> , <code>set</code> , <code>object</code> , <code>tuple</code> , <code>any</code>)	No (defaults to <code>any</code>)
<code>default</code>	Default value if none is provided	No (makes variable optional)
<code>sensitive</code>	If <code>true</code> , hides value in CLI output	No (default <code>false</code>)
<code>validation</code>	Block with <code>condition</code> and <code>error_message</code> for input validation	No
<code>nullable</code>	If <code>false</code> , variable cannot be set to <code>null</code>	No (default <code>true</code>)

Simple Variables File

Create a `variables.tf`:

```
variable "environment" {
  description = "Deployment environment (dev, qa, prod)"
  type       = string
  default    = "dev"
}

variable "bucket_name" {
  description = "Name of the S3 bucket"
  type       = string
}

variable "table_name" {
  description = "Name of the DynamoDB table"
  type       = string
  default    = "DefaultTable"
}

variable "queue_name" {
  description = "Name of the SQS queue"
  type       = string
}

variable "enable_versioning" {
  description = "Enable S3 bucket versioning"
  type       = bool
  default    = false
}

variable "tags" {
  description = "Tags to apply to all resources"
  type       = map(string)
}
```

```
default      = {}  
}
```

3. Variable Type Constraints

Primitive Types

```
variable "name"      { type = string }  
variable "count"     { type = number }  
variable "enabled"   { type = bool  }
```

Collection Types

```
variable "regions"    { type = list(string)  }  
variable "config"     { type = map(string)   }  
variable "unique_ids" { type = set(string)   }
```

list(string) — Ordered collection

```
# variables.tf  
variable "regions" {  
  description = "AWS regions for resource deployment"  
  type        = list(string)  
}  
  
# terraform.tfvars  
regions = ["us-east-1", "eu-west-1", "ap-southeast-1"]  
  
# main.tf – using list with count.index  
resource "aws_sqs_queue" "regional" {  
  count = length(var.regions)  
  name  = "queue-${var.regions[count.index]}"  
}
```

This creates one queue per region: **queue-us-east-1**, **queue-eu-west-1**, **queue-ap-southeast-1**.

set(string) — Unordered collection (no duplicates)

```
# variables.tf  
variable "unique_ids" {  
  description = "Unique identifiers (duplicates are silently dropped)"  
  type        = set(string)  
}
```

```
# terraform.tfvars
unique_ids = ["alpha", "beta", "gamma", "alpha"] # "alpha" appears only
once

# main.tf – using for_each with set
resource "aws_s3_bucket" "from_set" {
  for_each = var.unique_ids
  bucket   = "bucket-${each.key}"
}
```

This creates three buckets: `bucket-alpha`, `bucket-beta`, `bucket-gamma` (the duplicate `alpha` is ignored).

`map(string)` — Key-value collection

```
# variables.tf
variable "config" {
  description = "Configuration key-value pairs"
  type        = map(string)
}

# terraform.tfvars
config = {
  environment = "dev"
  project     = "myapp"
  owner       = "platform"
}

# main.tf – accessing map values
resource "aws_s3_bucket" "main" {
  bucket = "app-${var.config["environment"]}-data"

  tags = var.config
}
```

Tags become: `environment = "dev"`, `project = "myapp"`, `owner = "platform"`.

Structural Types

`object({...})` — Typed structured data

```
# variables.tf
variable "resource_config" {
  description = "Complete resource configuration object"
  type = object({
    name    = string
    size    = number
  })
}
```

```

    tags      = map(string)
    encryption = bool
    retention  = number
  })
}

# terraform.tfvars
resource_config = {
  name      = "my-bucket"
  size      = 100
  tags = {
    Environment = "dev"
  }
  encryption = true
  retention  = 30
}

# main.tf – accessing object fields
resource "aws_s3_bucket" "main" {
  bucket = var.resource_config.name
  tags   = var.resource_config.tags
}

resource "aws_s3_bucket_server_side_encryption_configuration" "main" {
  count = var.resource_config.encryption ? 1 : 0
  bucket = aws_s3_bucket.main.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_sqs_queue" "main" {
  name = "${var.resource_config.name}-queue"
  message_retention_seconds = var.resource_config.retention * 86400
}

```

Object fields are accessed with dot notation: `var.resource_config.name`.

`tuple([type, type, ...])` — Positional typed list

```

# variables.tf
variable "endpoint_config" {
  description = "Endpoint: [host, port, use_ssl]"
  type        = tuple([string, number, bool])
}

# terraform.tfvars
endpoint_config = ["api.example.com", 443, true]

```

```
# main.tf – accessing tuple by position
locals {
  endpoint_host = var.endpoint_config[0] # "api.example.com"
  endpoint_port = var.endpoint_config[1] # 443
  endpoint_ssl  = var.endpoint_config[2] # true
}

resource "aws_sqs_queue" "main" {
  name = "queue-${var.endpoint_config[0]}"
}
```

Tuple elements are accessed by index: `var.endpoint_config[0]`. Unlike lists, each position has a specific type.

4. Using Variables in Resources

Variables are referenced as `var.<variable_name>`:

```
resource "aws_s3_bucket" "main" {
  bucket = "${var.environment}-${var.bucket_name}"

  tags = var.tags
}

resource "aws_s3_bucket_versioning" "main" {
  bucket = aws_s3_bucket.main.id

  versioning_configuration {
    status = var.enable_versioning ? "Enabled" : "Suspended"
  }
}

resource "aws_dynamodb_table" "main" {
  name          = var.table_name
  billing_mode  = "PAY_PER_REQUEST"
  hash_key     = "pk"

  attribute {
    name = "pk"
    type = "S"
  }
}

resource "aws_sqs_queue" "main" {
  name = "${var.environment}-${var.queue_name}"
}
```

5. Passing Variable Values

There are multiple ways to set variable values, with this precedence (highest wins):

```
CLI flags (-var) > .tfvars file > Environment variables > Default value
```

Method 1: Default Values (simplest)

The `default` in the variable block is used when nothing else is provided.

Method 2: `terraform.tfvars` File

Create `terraform.tfvars`:

```
environment      = "dev"
bucket_name      = "my-app-data"
table_name       = "AppTable"
queue_name       = "event-queue"
enable_versioning = true
tags = {
  Environment = "dev"
  ManagedBy   = "Terraform"
}
```

Terraform automatically loads files named `terraform.tfvars` or `*.auto.tfvars`.

Method 3: Custom `.tfvars` File

```
# dev.tfvars
environment = "dev"
bucket_name = "my-app-data-dev"

# prod.tfvars
environment = "prod"
bucket_name = "my-app-data-prod"
```

Apply with:

```
terraform apply -var-file="prod.tfvars"
```

Method 4: CLI `-var` Flag

```
terraform apply -var="bucket_name=my-bucket" -var="environment=prod"
```

Method 5: Environment Variables

```
export TF_VAR_bucket_name="my-bucket"  
export TF_VAR_environment="prod"  
terraform apply
```

The prefix `TF_VAR_` tells Terraform this is a variable value.

6. Variable Validation

```
variable "environment" {  
  description = "Deployment environment"  
  type        = string  
  
  validation {  
    condition     = contains(["dev", "qa", "prod"], var.environment)  
    error_message = "Environment must be one of: dev, qa, prod."  
  }  
}  
  
variable "instance_count" {  
  description = "Number of instances"  
  type        = number  
  
  validation {  
    condition     = var.instance_count > 0 && var.instance_count <= 10  
    error_message = "Instance count must be between 1 and 10."  
  }  
}
```

7. Sensitive Variables

```
variable "db_password" {  
  description = "Database password"  
  type        = string  
  sensitive   = true  
}
```

When `sensitive = true`, the value is hidden in `terraform plan` and `terraform apply` output:


```
+ db_password = (sensitive value)
```

8. Complete Example

```
# variables.tf
variable "environment" {
  description = "Deployment environment"
  type        = string

  validation {
    condition     = contains(["dev", "qa", "prod"], var.environment)
    error_message = "Environment must be dev, qa, or prod."
  }
}

variable "bucket_name" {
  description = "Base name for S3 bucket"
  type        = string
}

variable "table_name" {
  description = "DynamoDB table name"
  type        = string
  default     = "AppData"
}

variable "queue_count" {
  description = "Number of SQS queues"
  type        = number
  default     = 1

  validation {
    condition     = var.queue_count >= 1 && var.queue_count <= 5
    error_message = "Queue count must be between 1 and 5."
  }
}

# terraform.tfvars
environment = "dev"
bucket_name = "my-app-storage"

# main.tf
resource "aws_s3_bucket" "main" {
  bucket = "${var.environment}-${var.bucket_name}"
}

resource "aws_dynamodb_table" "main" {
  name           = "${var.environment}-${var.table_name}"
  billing_mode   = "PAY_PER_REQUEST"
```

```
hash_key      = "pk"

attribute {
  name = "pk"
  type = "S"
}

resource "aws_sqs_queue" "main" {
  name = "${var.environment}-queue-${count.index}"
  count = var.queue_count
}
```

9. Key Takeaways

- Variables make configurations reusable across environments
- `var.<name>` is how you reference a variable in resources
- Variables can have type constraints, defaults, and validation rules
- Variable values come from multiple sources with a clear precedence order
- `terraform.tfvars` is auto-loaded; custom files need `-var-file`
- Environment variables use the `TF_VAR_` prefix
- `sensitive = true` hides values in logs and CLI output

example and exercise repo url

<https://github.com/eyu-se/terraform-training-material-with-projects>

Trainer – Eyuel M.